DESAFIO ENTREGA INTERMEDIÁRIA: IMPLEMENTAÇÃO COM IA, TEST HARNESS E VERSIONAMENTO NO GITHUB

Disciplina: Bootcamp III
Atividade: Desafio Entrega Intermediária (Atividade Cumulativa - Fase 2)
Carga Horária: 75 horas
Tipo de Trabalho: Individual
Modalidade: EAD - Disciplina Virtual (DV)
Valor Avaliativo: 15,0 pontos (15% da pontuação total da disciplina)

1. CONTEXTUALIZAÇÃO E OBJETIVO GERAL

Nesta etapa, você colocará em prática o ciclo de desenvolvimento assistido por Inteligência Artificial baseado no contrato técnico executável criado na Fase 1. Seu papel nesta fase evolui de "arquiteto de especificações" para desenvolvedor modernizador, piloto de agentes de IA e homologador crítico.

O objetivo desta atividade é implementar o sistema TaskAnalyzer gerando o código-fonte via assistentes de IA (ex.: Gemini, ChatGPT, GitHub Copilot, Claude) estritamente guiado pelas regras de contexto (CONTEXT_RULES.md) e pelo contrato de negócio (`task_analyzer_spec.md`), criar e executar o Test Harness automatizado (pytest) para validar o comportamento do sistema e versionar toda a solução em um repositório público no GitHub utilizando boas práticas de Git/GitHub (Branches, Commits e Pull Requests).

2. PROJETO CUMULATIVO - FLUXO DE CONTINUIDADE

- **FASE 1 (Concluída - 10,0 pts):** Especificação SDD no Google Docs, Cenários de Aceite (Dado-Quando-Então), Governança de Contexto (`CONTEXT_RULES`) e Vídeo Explicativo.
- **FASE 2 (ESTA ENTREGA - 15,0 pts):** Criação do repositório no GitHub, geração do código via IA com base na especificação, implementação do Test Harness (`pytest`), execução dos testes, versionamento via Git e abertura de Pull Requests (Atual) e Vídeo Explicativo.
- **FASE 3 (Entrega Final - 25,0 pts):** Automação CI/CD com GitHub Actions, Containerização (Docker), Relatório de Governança/Segurança (Análise de Alucinações e Ética) e Homologação Final.

3. DETALHAMENTO DO CONTEÚDO E REQUISITOS

3.1 Criação e Estruturação do Repositório no GitHub

Você deverá criar um repositório público no GitHub intitulado `sdd-task-analyzer` e materializar a estrutura de arquivos planejada na Fase 1:sdd-task-analyzer/

├─── README.md # Documentação geral do projeto, instruções de execução e status do build

├─── CONTEXT_RULES.md # Regras de governança de IA transpostas da Fase 1

├─── .gitignore # Arquivos/diretórios ignorados (ex.: __pycache__/, .pytest_cache/, venv/)

├─── requirements.txt # Dependências autorizadas (pytest, etc.)

├─── specs/

│ └─── task_analyzer_spec.md # Especificação do contrato SDD transposta da Fase 1

├─── tests/

│ └─── test_harness.py # Suíte de testes automatizados com pytest (Cenários 1 e 2)

└──── src/

    └──── task_analyzer.py   # Código-fonte do sistema gerado via IA e homologado por humano

3.2 Geração do Código via Assistentes de IA (AI Harness & SDD)

1. Forneça o conteúdo do `CONTEXT_RULES.md` e do `specs/task_analyzer_spec.md` como contexto para o seu assistente de IA.
2. Solicite à IA a geração do código em `src/task_analyzer.py` para a função `analyze_tasks` e para a exceção customizada `TaskValidationError`.
3. Garanta que o código gerado cumpra rigorosamente as diretrizes:
    - Python 3.11+;
    - *Type hints* em 100% das funções/parâmetros;
    - Tratamento de erro com exceções específicas e prevenção de divisão por zero.

3.3 Implementação do Test Harness (`pytest`)

1. No arquivo `tests/test_harness.py`, implemente a suíte de testes utilizando o framework `pytest`.
2. O Test Harness deve traduzir diretamente os cenários de aceite definidos na Fase 1:
    - **Teste de Sucesso (Cenário 1):** Validação dos cálculos de `total_tarefas`, `total_concluidas`, `total_pendentes`, `tempo_medio_conclusao_horas` (geral e por prioridade) e `taxa_atraso_percentual`.
    - **Teste de Exceção/Erro (Cenário 2):** Validação de datas inconsistentes (`data_conclusao < data_criacao`) disparando `TaskValidationError`.
    - **Teste de Casos de Borda:** Validação de entradas vazias ou apenas com tarefas pendentes, garantindo o retorno de `0.0` para médias sem lançar exceções não tratadas.
3. Execute o `pytest` no seu ambiente local ou terminal e garanta 100% de aprovação nos testes.

3.4 Versionamento e Fluxo de Trabalho com Git/GitHub

1. **Branching Strategy:** Trabalhe com a ramificação principal `main` e crie ramificações de funcionalidade (ex.: `feature/sdd-specification`, `feature/test-harness`, `feature/task-analyzer-impl`).
2. **Commits Atomicos:** Realize commits pequenos, frequentes e com mensagens descritivas (ex.: `feat: adiciona especificacao sdd e context rules`, `test: implementa test harness com pytest para cenario 1 e 2`, `feat: implementa algoritmo task_analyzer via assistente de IA`).
3. **Pull Requests (PRs):** Abra pelo menos **um Pull Request** documentado no GitHub relacionando a branch de funcionalidade à branch `main`, simulando o processo de homologação e revisão técnica humana.
4. **Vídeo de Apresentação Técnica:** Grave um vídeo de 5 minutos, na horizontal (Não pode ser Short), com áudio e câmera visíveis, postado no YouTube como "Não Listado" ou "Público". O vídeo deve demonstrar a estrutura do repositório, a execução dos testes no terminal (100% passed), o fluxo de Pull Request e a validação humana contra as `CONTEXT_RULES`.

4. FORMA DE SUBMISSÃO E ENTREGÁVEIS DA FASE 2

A submissão na plataforma salaonline deve ser realizada enviando um arquivo PDF individual contendo obrigatoriamente:

1. **Link Ativo do Repositório GitHub:** URL direta do repositório público `sdd-task-analyzer`.
2. **Relatório Técnico da Fase 2:**
   - Prints do terminal demonstrando a execução e aprovação dos testes automatizados com `pytest` (100% *passed*).
   - Print da árvore de arquivos no GitHub.
   - Print do Pull Request aberto/merged no GitHub.
   - Breve relato sobre os prompts utilizados e como as `CONTEXT_RULES` evitaram "alucinações" do assistente de IA.
3. **Link do Vídeo de Apresentação Técnica (YouTube):**
   - URL do vídeo demonstrando o repositório, execução do pytest, fluxo de PR e aderência às regras de contexto e contrato SDD.

5. CRITÉRIOS DE AVALIAÇÃO (BAREMA DE CORREÇÃO - FASE 2)

Critério	Descrição Detalhada	Pontuação
Estrutura do Repositório e Versionamento Git/GitHub	Repositório público criado corretamente, com árvore de arquivos completa (`specs/`, `tests/`, `src/`, `README.md`, `CONTEXT_RULES.md`), branches organizadas e ao menos 1 Pull Request revisado/documentado.	**4,0 pts**
Implementação e Aderência do Código (SDD & IA)	Código Python em `src/task_analyzer.py` gerado via IA em total conformidade com a especificação SDD e `CONTEXT_RULES` (Python 3.11+, type hints, tratamento de erros).	**4,0 pts**
Test Harness e Execução do Pytest	Suíte de testes em `tests/test_harness.py` cobrindo integralmente os Cenários 1 e 2 e casos de borda, com evidência (prints) de 100% de aprovação na execução do `pytest`.	**3,0 pts**
Vídeo de Apresentação Técnica (YouTube)	Vídeo de 5 min (horizontal, com áudio/câmera) demonstrando o	**3,0 pts**

Critério	Descrição Detalhada	Pontuação
	repositório, execução bem-sucedida do pytest, fluxo de PR e aderência às regras de governança e contrato SDD.	
Qualidade da Documentação e Submissão PDF	Envio do PDF na plataforma salaonline no prazo, contendo links válidos (GitHub, Docs, Vídeo), prints comprobatórios e relato de homologação de IA estruturado.	**1,0 pts**
TOTAL		**15,0 pts**